# Grand Finale: Chocolate Batch Plant, Build to Batch

Prepared for Shadow · 5 November 2026 · Lam Research Challenge 2026, Stage 3 · Team 13

**Contents**


- Overview
- P&ID
- Mechanical build
- Electrical wiring
- Firmware
- Commissioning and calibration
- Milestone 1: water dry run
- Milestone 2: Chocolate batch
- Troubleshooting
- Background: the ideas behind this guide
- Appendix: firmware source


## Overview

The Grand Finale is one build with two milestones. Your team has to design and build a table-top paint plant that doses three base colours into a mixing tank in a fixed order, then make about 3,000 mL of **Chocolate** that matches the judges' reference colour. The whole sequence must run from **one press of a single start switch**. This guide takes you from an empty table to the finished batch, in the order you should do things.

| Milestone | What the judges check | Where in this guide |
|---|---|---|
| 1 Design, build, dry run | P&ID on paper matches the build · layout built from the kit · water dry run doses each base line precisely | P&ID, Mechanical build, Electrical wiring, Firmware, Commissioning and calibration, Milestone 1 |
| 2 Achieve the target colour | Batch runs automatically from one start switch · about 3,000 mL in the mixing tank · colour matches the reference | Milestone 2 |

Recipe for a 3,000 mL batch, from the Chocolate task sheet:

| Line | Base colour | Share | Volume | Pump | Flow sensor | Line valve |
|---|---|---|---|---|---|---|
| 1 | Red | 60 % | **1,800 mL** | P1 | FT1 | SV1 |
| 2 | Yellow | 10 % | **300 mL** | P2 | FT2 | SV2 |
| 3 | Green | 30 % | **900 mL** | P3 | FT3 | SV3 |
| 4 | Flush water | none | chase plus flush | P4 | none | SV4 |

Fixed numbers used throughout:

- Pumps: EARTH 4002 diaphragm pumps, 12 V (9 to 14 V), 3.5 A, rated 5.5 L/min (about 90 mL/s) free flow. In a real line with a flow sensor, valves and 5 mm bore tube, expect much less. The similar Stage 2 pump managed about 2 L/min (about 33 mL/s), so plan on **25 to 40 mL/s** at full duty until you measure it. That makes Yellow about 10 s, Green about 30 s and Red about 60 s.
- Solenoid valves: Uflow, 12 V DC 10 W, **normally closed** (no power means shut), 1/4" BSP brass body, 0 to 10 bar, with a flow arrow cast on the body.
- Flow sensors: 3-wire Hall-effect pulse sensors of unknown model, so the pulses per mL ( `k` ) **must** be calibrated from scratch. A typical sensor of this size gives roughly 0.5 to 6 pulses per mL. Treat that only as a sanity check.
- Tube: clear PU 8 mm OD × 5 mm ID, which holds **19.6 mL per metre**. Blue tube 8 × 10 mm, which holds about 50 mL per metre.
- Power: 12 V 30 A SMPS. Only one pump ever runs at a time, plus at most one valve (about 0.8 A), so the peak load is about 6 A.

**Do it in this order:** draw the P&ID → mechanical build → leak test with water (no power) → electrical wiring → flash both ESP32s → I/O checks → prime → find each pump's slowest reliable duty → calibrate `k` for each flow sensor → trim with full test doses → water dry run batch (Milestone 1) → switch to colour → Chocolate batch (Milestone 2). Each step relies on the one before it. Calibrating before the dry run means the dry run passes first time.

# P&ID

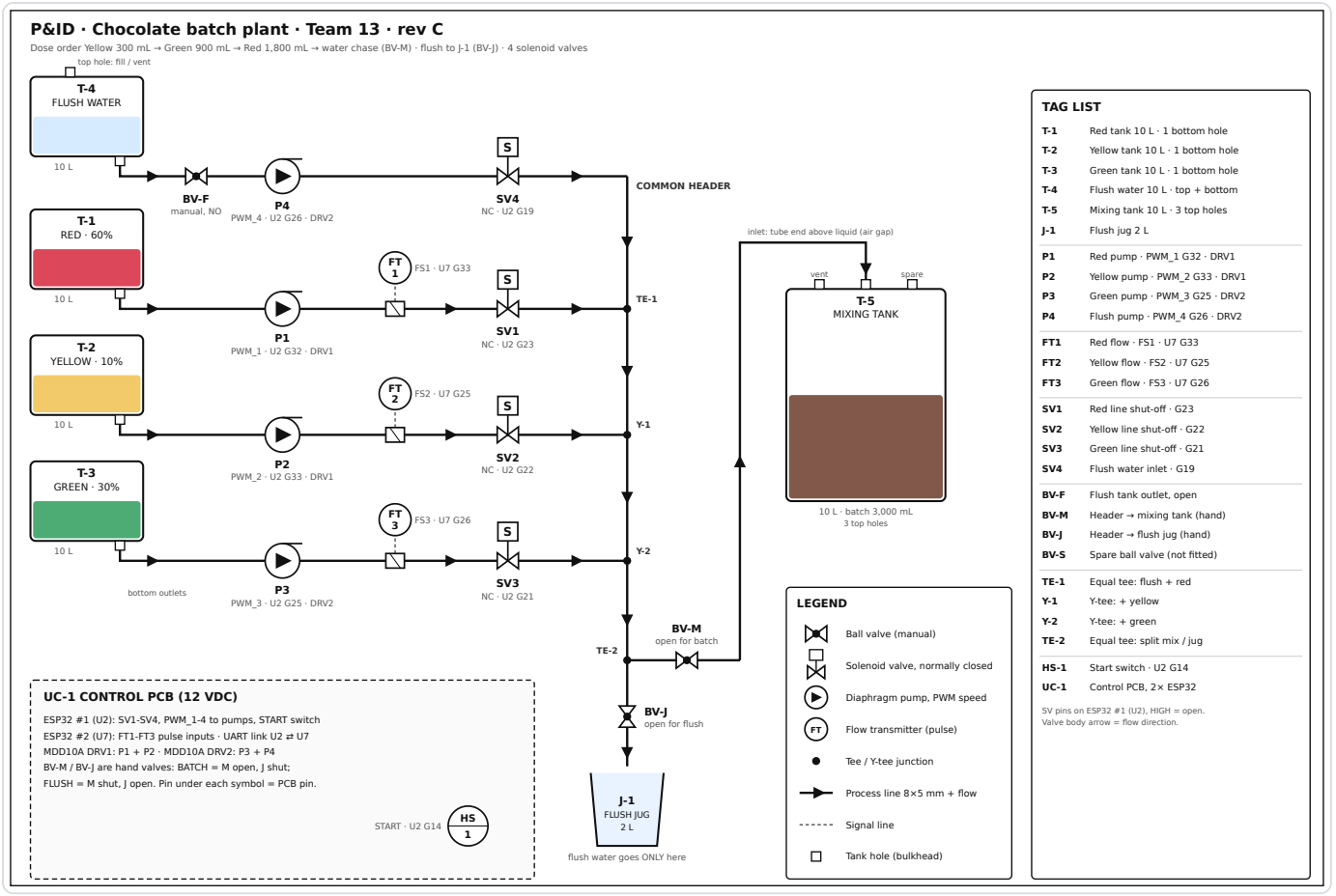



Figure 1. P&ID, Team 13 Chocolate batch plant. The pin printed under each symbol is where its signal lands on the PCB. It is also in /pid as chocolate-pid.png (printing) and .svg (scaling).

## How the plant works

- **Three identical colour lines.** Each runs from the tank's single bottom hole → pump Pn → flow sensor FTn → solenoid valve SVn → common header. The flow sensor sits on the pump's discharge, where the line is full and pressurised, so it does not see air bubbles from the suction side. The solenoid valve sits last, as close to the header as possible, so it cuts the dose off sharply.
- **One common header.** All lines join one vertical header: flush water enters at the very top through TE-1, Red joins at TE-1, Yellow at Y-1 and Green at Y-2. At the bottom, TE-2 splits it in two, and each branch has a hand ball valve: **BV-M** goes up into the middle top hole of the mixing tank and **BV-J** goes down into the 2 L flush jug.
- **Flush line.** T-4 water (bottom hole) → BV-F → P4 → SV4 enters at the *start* of the header, so flush water sweeps every junction on its way out. The colour valves SV1 to SV3 stay shut during a flush, so no water ever enters a colour line and no colour is ever diluted.
- **Why the colour lines themselves don't need flushing between batches:** each one only ever carries its own colour. The only section that sees more than one colour is the header, and that is exactly what the flush cleans.
- **Only 4 solenoid valves, so the outlet is switched by hand.** For the batch, BV-M is open and BV-J is shut, and the whole dose sequence runs from one START press. When the batch is done, the controller asks you to shut BV-M and open BV-J. A second START press then runs the flush, which

goes **only to the jug**, as the rules require. The one exception is a short timed water "chase" through BV-M straight after the last colour. It pushes the colour left in the header into the mixing tank so every metered drop reaches the batch. It adds about 1 to 2 % water, which does not visibly change the colour.

- **Interlocks in firmware:** a pump can only run when its own solenoid valve is open, and closing a valve stops its pump first. Every solenoid valve closes at power-up because they are normally closed. The firmware cannot see the hand valves, so every message says which way they must be set.
- **Tank holes used:** Red, Yellow and Green (1 bottom hole each) = outlet. Flush water tank (top + bottom) = bottom outlet, top hole for filling and venting. Mixing tank (3 top holes) = middle hole for the header inlet, one as an air vent, one spare.

## Tag list

| Tag | Item | Where it sits / PCB connection |
|---|---|---|
| T-1 | Red base tank, 10 L | Filled by the technical team; outlet at the bottom |
| T-2 | Yellow base tank, 10 L | Same as T-1 |
| T-3 | Green base tank, 10 L | Same as T-1 |
| T-4 | Flush water tank, 10 L | Clean water; outlet at the bottom |
| T-5 | Mixing tank, 10 L | Fed from the top; tube end above the liquid |
| J-1 | 2 L measuring jar (flush jug) | Under the BV-J outlet |
| BV-F | Ball valve, flush tank outlet | T-4 bottom hole. Open during runs |
| BV-M | Ball valve, header → mixing tank (hand) | TE-2 branch up to T-5. **Open for batch and chase**, shut for flush and calibration |
| BV-J | Ball valve, header → flush jug (hand) | TE-2 branch down to J-1. **Shut for batch**, open for flush, prime and calibration |
| BV-S | Fourth ball valve | Spare, not fitted |
| P1 / P2 | Red / Yellow pump | MDD10A **DRV1** channel 1 / 2 · PWM_1 G32 / PWM_2 G33 on ESP32 #1 |
| P3 / P4 | Green / Flush pump | MDD10A **DRV2** channel 1 / 2 · PWM_3 G25 / PWM_4 G26 on ESP32 #1 |
| FT1 / FT2 / FT3 | Flow sensors, Red / Yellow / Green | FLOW_SENSOR1 / 2 / 3 · FS1 G33 / FS2 G25 / FS3 G26 on ESP32 #2 |
| SV1 / SV2 / SV3 | Line shut-off, Red / Yellow / Green | SV1 G23 / SV2 G22 / SV3 G21 on ESP32 #1 |
| SV4 | Flush water inlet | SV4 G19 |
| TE-1 | Equal tee 8×8×8 | Top of header: flush + Red |
| Y-1, Y-2 | Y tees 8×8×8 | Header: + Yellow, + Green |
| TE-2 | Equal tee 8×8×8 | Bottom of header: split to BV-M / BV-J |

| Tag | Item | Where it sits / PCB connection |
|---|---|---|
| HS-1 | Tactile START switch | ON/OFF_SWITCH1 · G14 on ESP32 #1 |
| UC-1 | Control PCB, 2 × NodeMCU-32S | U2 = ESP32 #1 (actuators), U7 = ESP32 #2 (sensors), UART link |

## Drawing it on paper

- Use the same layout as Figure 1: tanks on the left, lines running left to right, the header in the middle, the mixing tank and jug on the right. It fits landscape A4 or A3.
- Symbols (ISA-5.1): a pump is a circle with a tangent discharge line. A ball valve is a bow-tie with a filled dot. A solenoid valve is a bow-tie with a stem and a square box marked S. A flow transmitter is a circle labelled FT n, joined to an inline box. Tees are dots on the line.
- Put a **flow arrow on every line**. Write "PU 8 × 5 mm" on the main lines and "blue 8 × 10 mm" at the pump and sensor stubs. Draw the controller as a dashed box (UC-1), and draw signal lines dashed.
- Write the tag list beside the drawing. Add four notes: "SV1 to SV4 normally closed", "BV-M open / BV-J shut for batch; reversed for flush", "flush water discharged only to J-1 (except the timed chase)" and "dose order Yellow → Green → Red". Draw each tank hole as a small square on the tank outline, in its real position (top or bottom).
- Before Milestone 1, walk the table with the paper in hand and tick every tag. The judges check that the drawing matches the build, so if you change a tube route, change the drawing too.

# Mechanical build

Build and leak test the whole water side **before** any electronics are connected. Wear goggles and gloves. Teflon tape goes on every threaded joint, with 4 to 6 wraps clockwise as you look at the thread end.

## What each fitting is for

The counts in the kit match this plan exactly, which is a good sign it is the intended use. Check the parts in your hand before you cut anything.

| Kit item | Qty | Use in this build |
|---|---|---|
| Male connectors 8 mm (1/4" BSP male × 8 mm push-in) | 20 | 4 solenoid valves × 2 ports = 8, plus 3 fitted ball valves × 2 ports = 6. 6 spare. |
| Hose connection 1/4" × 8 mm, nuts 1/4", washers | 8 / 4 / 37 | One through each of the 8 tank holes (3 colour bottoms, flush top and bottom, 3 mixing-tank tops), with a washer each side of the wall. Use the 4 nuts on the 4 bottom holes, where water pressure acts. |
| Female connectors 8 mm | 4 | Secure the hose connections in the top holes (no pressure there), or join a ball valve to a bulkhead if the threads need it. |
| PU connectors 8 × 10 mm (reducers) | 14 | Join the clear 8 mm tube to the short blue 10 mm stubs on pump and sensor barbs: 4 pumps × 2 + 3 sensors × 2 = 14. |
| Tube clamps for 8 mm | 15 | One on every blue stub over a barb (14), plus 1 spare. |
| PU connectors 8 × 8 mm | 3 | Straight joiners for any tube that ends up too short. |
| Equal tees 8×8×8 / Y tees 8×8×8 | 2 / 2 | TE-1, TE-2 / Y-1, Y-2 on the header. |
| Blue tube 8 × 10 mm | 2 m | Short stubs (about 5 to 8 cm) pushed over pump and flow sensor barbs. |
| Clear tube 8 × 5 mm | 20 m | Everything else. |

## Layout on the table

1. **Tanks:** T-1 Red, T-2 Yellow, T-3 Green and T-4 Water in a row at the back. T-5 Mixing tank at the front with the jug J-1 beside it. Put the supply tanks on the same level as the pumps or slightly higher, so the pumps self-prime easily.
2. **Tank holes:** fit a hose connection through every hole. The colour tanks' bottom holes go straight to their pumps. The flush tank's bottom hole goes to BV-F (handle in line with the tube means open). Its top hole stays open as a fill and vent hole. On the mixing tank, the middle top hole takes the header inlet and one other stays open as a vent.
3. **Pumps:** screw them down in a row in front of the tanks. The inlet faces the tank. Check the arrows or labels on the pump head: a pump plumbed backwards will not pump.
4. **Flow sensors:** right after each colour pump, **arrow pointing away from the pump**. Mount them level, with the wires on top.

5. **Solenoid valves:** after each sensor, **body arrow pointing toward the header**. These are pilot-type brass valves and only seal in the arrow direction. Mount the coil upright.
6. **Header:** join the four lines with TE-1, Y-1, Y-2 and TE-2 as in the P&ID, routed neatly with no sags where air can collect. Measure the header length from TE-1 to the end of the tube inside T-5 and write it down. The water chase time is set from that volume (19.6 mL per metre).
7. **Mixing tank branch:** TE-2 → BV-M → up to the middle top hole of T-5. The tube end inside must stay **above** the liquid (about 4 cm above the 3 L mark), so colour can never siphon back.
8. **Jug branch:** TE-2 → BV-J → short tube pointing straight down into J-1, so it drains completely after every run. That keeps calibration doses repeatable. Put BV-M and BV-J where you can reach both handles at once, and label them "M" and "J".
9. Label every component with its tag (masking tape and a marker). It helps the judges and saves time when debugging.

## Leak test (no power)

- Put 2 to 3 L of water in each supply tank (water, not colour; colour comes in Milestone 2). Open BV-F. The solenoid valves are closed with no power, so water fills each line up to its SV.
- Wait 5 minutes and check every joint with a dry tissue. Re-tape or re-seat anything that weeps. A drip under a pump is usually a tube not pushed fully over the barb or a missing clamp.
- Check that no water reaches the header. If it does, a solenoid valve is in backwards. NC valves only hold pressure in their arrow direction.

# Electrical wiring

> **SMPS off at the wall for every wiring change. Never plug or unplug connectors with power on.** Keep 12 V wiring above and away from the wet side. Check polarity with a multimeter before the first power-on.

## Connector by connector

| PCB connector | Connects to | Notes |
|---|---|---|
| 12V INPUT (screw terminal) | SMPS +V / −V | Polarity checked with a meter. The fused input feeds Vin. The blue LED (LED10) shows 12 V is present. |
| P1 (12 V out) | MDD10A **DRV1** power terminal (+ / −) | Powers pumps 1 and 2. |
| P2 (12 V out) | MDD10A **DRV2** power terminal (+ / −) | Powers pumps 3 and 4. P3 is spare because there is no third driver. |
| DRV1 motor 1 (M1A / M1B) | **P1 Red pump** red / black | If the pump hums but does not move water, swap the two leads. |
| DRV1 motor 2 (M2A / M2B) | **P2 Yellow pump** | |
| DRV2 motor 1 (M1A / M1B) | **P3 Green pump** | |
| DRV2 motor 2 (M2A / M2B) | **P4 Flush pump** | |
| MOTOR_DRIVER1 (6-pin) | DRV1 logic input header | PCB pin 2 = PWM_1 (G32), pin 4 = PWM_2 (G33), pins 1, 3, 5 and 6 = GND. Pins 1 and 3 hold DIR1 and DIR2 low, so each pump turns one way only. Match the cable to the **labels** printed on the MDD10A (PWM1, DIR1, PWM2, DIR2, GND). Do not go by position alone. |
| MOTOR_DRIVER2 (6-pin) | DRV2 logic input header | Pin 2 = PWM_3 (G25), pin 4 = PWM_4 (G26), the rest as above. |
| MOTOR_DRIVER3 | nothing | Unused. |
| SV1 to SV4 valve outputs (2-pin) | Valve coils SV1 Red, SV2 Yellow, SV3 Green, SV4 Flush; yellow and black leads | Pin 1 = +12 V, pin 2 = switched −. The coils have no polarity. The on-board MOSFET module switches each coil when its ESP32 #1 pin goes HIGH. |
| FLOW_SENSOR1 / 2 / 3 (S V G) | FT1 Red / FT2 Yellow / FT3 Green | V = 5 V (5V1), G = SGND, S = signal to ESP32 #2. The usual colours are red = V, black or brown = G and the third wire (yellow or orange) = S. Confirm during priming (Commissioning step 2): only the count of the line that is running should rise. FLOW_SENSOR4 and 5 are unused. |

| PCB connector | Connects to | Notes |
|---|---|---|
| ON/OFF_SWITCH1 (2-pin) | Tactile switch HS-1 | Use two diagonal legs, so pressing the switch connects them. It is read on G14 with a pull-up, and pressing it pulls the pin to SGND. |
| FLOAT_SWITCH, TEMPERATURE, COLOUR, HEATER | nothing | Not in this kit and not used. |
| USB | Laptop | The kit has one ESP32 cable: flash one board, then move the cable to the other. Leave it on ESP32 #1 for runs, because that is where the log and commands are. |

## Before first power-on

- Meter the SMPS output (about 12.0 to 12.5 V) and its polarity at the PCB terminal.
- With the SMPS on and no USB connected, both ESP32 power LEDs should light and LED10 (12 V) should be on. No valve should click, because valve pins are LOW at boot.
- Measure the flow sensor S pin against SGND while nothing flows. It should read 0 V or 3.3 V. If it reads a steady 5 V, tell the technical team before running for long, because ESP32 inputs are 3.3 V parts.
- Pumps never run dry and never against a closed valve. The firmware enforces this, but do not override it with manual commands.

# Firmware

The project is `firmware/chocolate_plant` (also zipped as `chocolate_plant.zip`). It is one PlatformIO project with two build environments, one per ESP32:

| Environment | Board | What it does |
|---|---|---|
| `controller` | ESP32 #1 (U2) | Owns every valve and pump, the START switch, the batch sequence, calibration commands and the serial log. Settings live in flash. |
| `sensor` | ESP32 #2 (U7) | Counts pulses from FT1 to FT3 with interrupts and sends the running totals to U2 every 10 ms over the on-board UART (G17 TX → G16 RX). It drives nothing. |

## Flashing

1. Copy `chocolate_plant` to the laptop and open the folder in VS Code with the PlatformIO extension (File → Open Folder).
2. The first build downloads the ESP32 platform (espressif32 6.9.0, Arduino core 2.0.17), so do it on Wi-Fi before the event if you can.
3. Plug USB into **ESP32 #2 (U7)**. In the PlatformIO sidebar, choose `env:sensor` → Upload. In the terminal: `pio run -e sensor -t upload`.
4. Move USB to **ESP32 #1 (U2)**: `pio run -e controller -t upload`, then `pio device monitor` (115200 baud).
5. If the upload stalls at "Connecting...", hold the board's BOOT button until it starts writing.
6. Type `help` in the monitor. In PlatformIO's monitor, type the command and press Enter. If the monitor echoes nothing, that is normal.

## How a dose works

Every metered dose, whether part of the batch or a test, follows the same profile:

1. Open the line valve SVn (the hand valves must already point the right way: BV-M for the batch, BV-J for tests). Wait 200 ms so the valve is fully open before the pump pushes.
2. **Fast phase:** pump at `fast` duty (default 100 %) until the metered volume reaches `target − band`. The band is 10 % of the target, but never less than 20 mL. For Yellow that is 270 mL.
3. **Slow phase:** drop to `slow` duty (default 35 %, set from your own test) for the last 10 %. This is the task sheet's tip, done automatically.
4. **Early stop:** cut the pump at `target − overrun`. The overrun is the volume that still passes the sensor after the stop command (pump spin-down plus link delay). After 150 ms the line valve closes.
5. **Settle and learn:** keep counting for about 1 s, report the final metered volume, and update the overrun estimate (the average of the old value and this coast), so each dose gets closer than the last.

Safety stops: no pulses within 4 s of starting (empty tank, air lock or shut ball valve), pulses stopping for 2 s mid-dose, lost data from U7, or any dose over 4 minutes. Each one stops every pump, closes every valve and prints the reason. Pressing START during a run aborts it.

## Batch sequence (one press of START)

| Step | Hand valves | Solenoids open | Pump | Ends when |
|---|---|---|---|---|
| 1 Yellow dose | BV-M open, BV-J shut | SV2 | P2 fast → slow | 300 mL metered by FT2 |
| 2 Green dose | same | SV3 | P3 fast → slow | 900 mL metered by FT3 |
| 3 Red dose | same | SV1 | P1 fast → slow | 1,800 mL metered by FT1 |
| 4 Water chase | same | SV4 | P4 at `p4` % | `chase` ms (pushes the header contents into T-5) |
| 5 Report, then wait | you now shut BV-M and open BV-J | none | none | Report printed; waits for the next START press |
| 6 Flush (2nd START press) | BV-M shut, BV-J open | SV4 | P4 at `p4` % | `flush` ms to the jug, refused if the jug would overflow |

The colour batch itself (steps 1 to 4) runs from **one** START press with no other action. The flush is a separate cleaning step afterwards. Before the next batch, set the hand valves back: BV-J shut, BV-M open.

Why Yellow → Green → Red: you can add colour but never remove it, so the plant doses the lightest colour first and the darkest last. Yellow is also the smallest and most sensitive dose, so it goes in first while everything is fresh and you can watch it. Each colour pushes the previous colour's leftover out of the header into the tank, and the chase does the same for Red. Change the order with `set order 231` (line numbers in dose order).

## Serial commands (ESP32 #1)

| Command | Does |
|---|---|
| `start` / `abort` | Run or stop the full batch. The same as pressing START. |
| `status` | Link health, raw pulse counts, every valve and pump state, and the START switch state. |
| `show` | All settings. |
| `prime n sec` | Run line n (1 to 4) into the **jug** at fast duty. Use it to fill lines and purge air. |
| `calrun n sec [duty]` | Timed run of line n into the jug that counts pulses. Then weigh the jug. |
| `k n mL` | Sets k (pulses per mL) = pulses of the last calrun or dose on line n ÷ the mL you measured. |
| `dose n mL [mix]` | One metered dose with the full fast/slow profile. Goes to the jug unless you add `mix`. |
| `chase` / `flush` | Timed water into the mixing tank / to the jug. |
| `jug mL` | Sets the jug estimate. Type `jug 0` after emptying it. |

| Command | Does |
| --- | --- |
| `valve n 0\|1`, `pump n %`, `off` | Manual tests (interlocked) and everything off. |
| `set key value` | Keys: `k1..3 tgt1..3 fast1..3 slow1..3 ovr1..3 order band bandmin p4 p4rate chase flush learn` |
| `save` / `load` / `defaults` | Settings in flash. **Type save after calibrating**, or a reset loses your work. |

While a pump runs, the controller prints a CSV line every 250 ms: `D,t_ms,line,phase,duty%,pulses,mL,mL_per_s`. Copy it from the monitor into a spreadsheet to plot volume and flow against time for each line.

> The code was compiled with a host C++ compiler against stub Arduino headers, and run through a simulated plant: a full batch, calibration and an empty-tank fault. It has **not** been compiled for the ESP32 itself, because the PlatformIO package server was blocked where it was written. Run `pio run` once as soon as you have the folder, and report any compiler message.

# Commissioning and calibration

All of this runs with **water** in all four supply tanks, the mixing tank empty, the jug empty and on the scale. A gram of water is a millilitre, so the scale is your reference. Put the jug on the scale and tare it before every weighing.

> For every step in this section, set the hand valves to the jug: **BV-M shut, BV-J open**. BV-F (flush tank outlet) stays open.

## Step 1: I/O check

1. On ESP32 #1, type `status`. It should say `link OK` with the ok count rising and bad near 0. If the link is down, check that U7 is flashed and powered.
2. Press START with no pumps armed: `status` shows `start=PRESSED` while you hold it. (With calibration missing, a press just prints "not calibrated, batch refused", which is fine.)
3. Valve click test: `valve 1 1` then `valve 1 0`, and the same for 2 to 6. Each should click. Note any valve that clicks for the wrong number and swap its connector.

## Step 2: Prime every line

- `prime 2 5`, `prime 3 5`, `prime 1 5` and `prime 4 5`. Each opens its SV and runs its pump into the jug. Repeat until water flows into the jug steadily with no bubbles in the clear tube.
- No flow after 4 s gives a "no flow after start" fault. That means a ball valve is shut, the pump is reversed (swap motor leads), the pump is plumbed backwards, or there is an air lock (lift the tube to let air out).
- Check that FTn's count rises only when line n runs. If FT2 counts while line 1 runs, two sensor connectors are swapped.
- Empty the jug and type `jug 0`.

## Step 3: Find the slow duty for each pump

The slow phase only helps if the pump still moves water reliably. For each line n:

1. `calrun n 5 30` (5 s at 30 %). If the report shows 0 pulses or a fault, the pump stalled at 30 %. Try 40 %, 50 % and so on.
2. Take the lowest duty that gives steady pulses, add 10 percentage points for margin, and set it: `set slow n …` (for example `set slow2 40`).
3. Write down the Hz printed at fast duty and at slow duty. Hz × 1000 ÷ k gives mL/s once k is known.

## Step 4: Calibrate k (pulses per mL)

Do this for each colour line n = 1, 2, 3:

1. Empty the jug, put it on the scale and tare.
2. `calrun n 15` (15 s at fast duty). The controller prints the pulse count.
3. Weigh the jug in grams (= mL), then type `k n <grams>`.

4. Repeat 3 times and fill in the table. If the three k values agree within about 2 %, set the average with `set k n …`. If one is far off, the jug probably overflowed or the tube was not full; redo it.
5. Type `save`.

| Line | Trial | Pulses N | Weighed mL | k = N / mL | Hz |
|---|---|---|---|---|---|
| 1 Red | 1 | | | | |
| 1 Red | 2 | | | | |
| 1 Red | 3 | | | | |
| 2 Yellow | 1 | | | | |
| 2 Yellow | 2 | | | | |
| 2 Yellow | 3 | | | | |
| 3 Green | 1 | | | | |
| 3 Green | 2 | | | | |
| 3 Green | 3 | | | | |

What to expect: the pulses per mL are roughly constant, typically 0.5 to 6. At low flow, turbine sensors often read a little low. That is why step 5 trims k using the real fast-then-slow dose profile instead of trusting the 15 s run alone.

## Step 5: Trim with real doses

Now test exactly what the batch will do, but into the jug:

1. Empty and tare the jug. `dose 2 300`. Read the report line (metered mL and coast), then weigh.
2. If the weighed volume differs from 300 by more than 1 % (3 mL), type `k 2 <weighed>`. That recalculates k from this dose's pulses. Repeat the dose until two in a row land within ±1 %.
3. Do the same with `dose 3 900` and `dose 1 900`. Red's 1,800 mL fills the 2 L jug almost to the brim, so trim Red with 900 mL doses, or do a final `dose 1 1800 mix` and weigh the mixing tank if your scale can take it.
4. Watch the "next overrun" figure settle after 2 or 3 doses. It learns how much arrives after the stop command.
5. `save`.

| Line | Target mL | Metered mL | Weighed mL | Error % | k after trim |
|---|---|---|---|---|---|
| 2 Yellow | 300 | | | | |
| 2 Yellow | 300 | | | | |
| 3 Green | 900 | | | | |
| 3 Green | 900 | | | | |
| 1 Red | 900 | | | | |
| 1 Red | 900 | | | | |

## Step 6: Flush pump rate, chase and flush times

1. Empty and tare the jug. `prime 4 10`, weigh, then `set p4rate <grams ÷ 10>`. (prime 4 runs at the `p4` duty, 60 % by default.)
2. Header volume $V_h$ = header tube length (TE-1 down to TE-2 and up to the end inside T-5, in m) × 19.6 mL. Typically 10 to 30 mL.
3. Chase time: `set chase` to 2 × $V_h$ ÷ p4rate × 1000 ms. For example, 15 mL and 30 mL/s gives 1000 ms. This pushes the last colour out of the header with a little margin.
4. Flush time: `set flush` so that 4 to 6 header volumes go to the jug. Watch the clear header tube during a flush; the water leaving should be clear by the end. 3000 to 6000 ms is typical. Each flush also adds to the jug estimate, and the controller refuses a flush that would overfill the 2 L jug.
5. `save`.

# Milestone 1: water dry run

1. All four supply tanks have water. The mixing tank is empty. The jug is empty, and you have typed `jug 0`.
2. Type `show` and take a photo of it, so you have a record of the calibration.
3. Set the hand valves for the batch: **BV-M open, BV-J shut**.
4. Press **START** once. Watch it run Yellow → Green → Red → chase → report with no further input, then show "BATCH COMPLETE".
5. Shut BV-M, open BV-J and press START again to run the flush into the jug. Then set the hand valves back: BV-J shut, BV-M open.
6. Copy the BATCH REPORT. It should show each line within about ±1 % of target and the ratios at 10.0 / 30.0 / 60.0.
7. Confirm the real total: weigh the mixing tank, or pour it into the jug 2 L at a time. Expect about 3,000 mL plus the chase water.
8. Show the judges each line's dose in the report and, if they ask, a single `dose n …` into the jug weighed on the scale.
9. **Empty and dry the mixing tank afterwards**, because any water left in it dilutes the colour batch.

☐ P&ID drawn on paper with tags, arrows and notes, and matching the build
☐ Build made only from kit parts, every component labelled with its tag
☐ Leak-free for 5 minutes at rest and during a run
☐ Both ESP32s flashed, link OK, valves and pumps on the right numbers
☐ k, slow duty and overrun calibrated for each colour line and saved
☐ Single test doses within ±1 % on the scale
☐ Full water batch from one START press, report saved, then flush to jug only

# Milestone 2: Chocolate batch

## Switching the lines from water to colour

1. Ask the technical team to empty the water from T-1 to T-3 and fill them with Red, Yellow and Green as on your task sheet. **Stir each base tank** before you start, because acrylic colour settles.
2. The colour lines are still full of water, which must go before you dose. With BV-M shut and BV-J open, run `prime n 3` for each line, repeatedly, into the jug until the clear tube *after the SV* shows solid colour all the way to the header. Line volume is roughly the tube length × 19.6 mL plus about 20 mL for the pump and blue stubs, so expect 50 to 100 mL per line.
3. Run `flush` (hand valves still to the jug) to clear the colour you just pushed into the header. Empty the jug and type `jug 0`.
4. Optional but worth it: `dose 2 100` into an empty, tared jug and read the volume on its scale markings. If it is more than about 3 % off, colour has changed the sensor's k compared with water. Correct it with `k 2 <mL>`, then flush. Colour is slightly denser than water, so trust the jug's mL markings over grams here.
5. Confirm the mixing tank is empty and dry.

## Running the batch

1. Type `show` and confirm the targets are tgt1 1800, tgt2 300, tgt3 900 and the order is 231.
2. Set the hand valves for the batch: **BV-M open, BV-J shut**. Check them twice: with BV-J open, colour goes to the jug.
3. Press **START** once. Do not touch anything until "BATCH COMPLETE".
4. Copy the BATCH REPORT. Stir the mixing tank (ask the judges first if stirring by hand is allowed) and present it next to the reference colour.
5. Shut BV-M, open BV-J and press START again to flush the header into the jug, so the next batch starts clean.

☐ Base tanks stirred, lines primed with colour, header flushed, jug emptied
☐ Mixing tank empty and dry before START
☐ Batch runs automatically from one START press
☐ About 3,000 mL in the mixing tank
☐ Colour compared with the reference by the judges
☐ Lines flushed with water into the 2 L jug only

# Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| "no data from sensor board U7" | U7 not flashed, not powered, or UART wiring | Flash env:sensor. `status` should show link OK. |
| "no flow after start" | BV-M and BV-J both shut, BV-F shut (flush line), tank empty, pump reversed or air lock | Open the right hand valve, refill, swap motor leads, or prime again. |
| "flow stopped" mid-dose | Tank ran dry or slow duty too low | Refill, or raise `slown` by 10. |
| Pump hums but no flow | Motor reversed (DIR fixed low) or pump plumbed backwards | Swap M+ and M− at the driver, or turn the pump round. |
| FTn counts when a different line runs | Flow sensor connectors swapped | Swap the FLOW_SENSOR plugs. |
| Wrong valve clicks | SV connectors swapped | Swap the plugs. Check with `valve n 1`. |
| Doses consistently high or low | k off | Trim with `dose` + `k n weighed`. |
| Doses scatter ±3 % or more | Air in the line, or slow duty near stall | Prime longer, raise slow duty, check the sensor is level and full. |
| Colour still in the header after flush | Flush too short | Raise `flush`. |
| ESP32 resets when a pump starts | Supply dip or noise | Check the SMPS rating and ground tie. Twist the pump leads. |
| Upload fails at "Connecting..." | Board not in download mode | Hold BOOT while uploading. |

# Background: the ideas behind this guide

## 1. Why the doses must be exact

Mixing paint is subtractive: every pigment absorbs part of the light, so each addition makes the mix darker. Pigment can be added but never removed, so an overshoot means a rejected batch. Yellow is only 10 % of the recipe, so an error in Yellow shifts the colour most. If Yellow lands 15 mL high (5 % of its dose), Yellow becomes 10.5 % of the batch, which is a visible hue shift. The same 15 mL on Red is under 1 %. That is why Yellow gets the gentlest treatment: dosed first, with a long slow band compared with its size.

## 2. Flow sensors and k

A Hall-effect flow sensor has a small turbine. Each time a magnet on the rotor passes the Hall chip, the output pin is pulled low, which gives one pulse. The number of pulses is proportional to the volume that has passed:

```
V (mL) = N (pulses) ÷ k (pulses per mL)
Q (mL/s) = f (Hz) ÷ k
```

k depends on the sensor's internal geometry and a little on the flow rate and the liquid's viscosity. That is why you calibrate against a scale (1 g of water = 1 mL), and trim using the same fast-then-slow profile the batch uses. Counting is done on a separate ESP32 with interrupts, so no pulse is missed while the controller is busy.

## 3. Why slow down near the target

After the controller decides to stop, some liquid still passes. Sensor data is up to 10 ms old, the loop reacts within a few ms, the pump takes tens of ms to spin down, and the line valve closes 150 ms later. The volume that slips through is roughly flow × reaction time:

```
overshoot ≈ Q × t_react
at full speed: 35 mL/s × 0.1 s ≈ 3.5 mL (1.2 % of Yellow)
at slow duty: 8 mL/s × 0.1 s ≈ 0.8 mL (0.3 % of Yellow)
```

Slowing for the last 10 % cuts the uncertainty several times over, and the learned *overrun* removes most of what is left. The controller stops early by the amount it has learned usually arrives late.

## 4. Dead volume in the header

The 5 mm bore tube holds π × (2.5 mm)² × 1000 mm = 19.6 mL per metre. Anything sitting in the shared header after a dose was metered but has not reached the tank. Two features handle it. First, the next colour pushes the previous colour's leftover into the tank. Second, after the last colour, the timed water chase pushes Red's leftover in. The cost is the header's worth of water from the previous flush (pushed in by Yellow) plus the chase water. That is about 1 to 2 % of the batch, which dilutes all three colours equally and does not change the ratios.

## 5. Normally closed valves and interlocks

A normally closed solenoid valve is shut until its coil is powered. On power loss, an ESP32 reset or a firmware crash, every line closes by itself and nothing siphons into the batch. These Uflow valves are pilot-operated and only seal in their arrow direction, which is why the arrow must point toward the header. The firmware adds interlocks on top. A pump never starts unless its path is open, closing a valve first stops the pump feeding it. The two hand valves are the one thing it cannot check, so its messages always say how they must be set. That protects the diaphragm pumps, which would otherwise build up to their 120 psi cutoff against a closed valve.

## 6. PWM speed control

The MDD10A switches the 12 V supply to the pump on and off 20,000 times a second. The fraction of time it is on (the duty) sets the average voltage and so the pump speed. Below a certain duty the motor cannot overcome the diaphragm's friction and stalls, which is why step 3 finds the lowest reliable duty before it is used for the slow phase. DIR is tied low on the PCB, so each pump only turns one way.

# Appendix: firmware source

## platformio.ini

```
; Team 13 · Chocolate batch plant · Lam Research Challenge 2026 Stage 3
;
; One project, two boards:
;   env:controller -> ESP32 #1 (U2): valves, pumps, start switch, batch sequence
;   env:sensor     -> ESP32 #2 (U7): counts the three flow sensors, streams counts to U2
;
; Upload:   pio run -e controller -t upload --upload-port <U2 port>
;           pio run -e sensor     -t upload --upload-port <U7 port>
; Monitor:  pio device monitor -e controller --port <U2 port>

[platformio]
default_envs = controller, sensor

[env]
platform = espressif32@6.9.0
board = nodemcu-32s
framework = arduino
monitor_speed = 115200
monitor_filters = direct
build_flags = -Wall

[env:controller]
build_src_filter = +<controller/>

[env:sensor]
build_src_filter = +<sensor/>
```

## include/link.h

```
// Shared definitions for the UART link between ESP32 #2 (sensor, U7) and ESP32 #1 (controller, U2).
//
// U7 sends one line every LINK_PERIOD_MS:
//     F,<seq>,<count1>,<count2>,<count3>*<xx>\n
// count1..3 are the running pulse totals of FS1..FS3 since U7 booted (uint32, they never reset).
// <xx> is the XOR of every character between 'F' and '*' inclusive of 'F', as two hex digits.
// U2 keeps its own baseline, so a lost frame never loses pulses: the next frame carries the total.
#pragma once
#include <Arduino.h>

static const uint32_t LINK_BAUD      = 115200;
static const uint8_t  LINK_RX_PIN    = 16;   // both boards: RX on G16
static const uint8_t  LINK_TX_PIN    = 17;   // both boards: TX on G17
static const uint32_t LINK_PERIOD_MS = 10;   // 100 frames per second

inline uint8_t linkChecksum(const char *s, size_t n) {
  uint8_t x = 0;
```

```
  for (size_t i = 0; i < n; i++) x ^= (uint8_t)s[i];
  return x;
}
```

## src/sensor/main.cpp (ESP32 #2)

```
// ESP32 #2 (U7) · sensor board
// Counts pulses from flow sensors FS1..FS3 and streams the running totals to ESP32 #1 over UART.
// It never drives anything, so it is safe to leave running all the time.
//
// USB serial commands (115200):
//   show      print the three running totals once
//   watch     toggle a 2 Hz print of totals and pulse rates (handy when wiring/priming)

#include <Arduino.h>
#include "link.h"

// PCB sheet 3: FLOW_SENSOR1..3 signal pins on ESP32 #2. (FS4 = G14, FS5 = G27 are unused.)
static const uint8_t FS_PIN[3] = {33, 25, 26};

static volatile uint32_t pulses[3] = {0, 0, 0};
static volatile uint32_t lastEdgeUs[3] = {0, 0, 0};
// Ignore edges closer than this (contact bounce / noise). 300 us = 3.3 kHz, far above any real flow
static const uint32_t MIN_EDGE_US = 300;

static void IRAM_ATTR countEdge(uint8_t i) {
  uint32_t now = micros();
  if (now - lastEdgeUs[i] >= MIN_EDGE_US) {
    pulses[i]++;
    lastEdgeUs[i] = now;
  }
}
static void IRAM_ATTR isr0() { countEdge(0); }
static void IRAM_ATTR isr1() { countEdge(1); }
static void IRAM_ATTR isr2() { countEdge(2); }

static portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
static void snapshot(uint32_t out[3]) {
  portENTER_CRITICAL(&mux);
  for (int i = 0; i < 3; i++) out[i] = pulses[i];
  portEXIT_CRITICAL(&mux);
}

static uint32_t seq = 0;
static bool watch = false;

void setup() {
  Serial.begin(115200);
  Serial2.begin(LINK_BAUD, SERIAL_8N1, LINK_RX_PIN, LINK_TX_PIN);
  for (int i = 0; i < 3; i++) pinMode(FS_PIN[i], INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FS_PIN[0]), isr0, FALLING);
  attachInterrupt(digitalPinToInterrupt(FS_PIN[1]), isr1, FALLING);
```

```
  attachInterrupt(digitalPinToInterrupt(FS_PIN[2]), isr2, FALLING);
  Serial.println(F("\n[U7 sensor] FS1=G33 FS2=G25 FS3=G26 -> UART G17 @115200. Commands: show, watch
}

void loop() {
  static uint32_t nextFrame = 0, nextWatch = 0;
  static uint32_t prevWatch[3] = {0, 0, 0};
  uint32_t now = millis();

  if ((int32_t)(now - nextFrame) >= 0) {
    nextFrame = now + LINK_PERIOD_MS;
    uint32_t c[3];
    snapshot(c);
    char buf[64];
    int n = snprintf(buf, sizeof(buf), "F,%lu,%lu,%lu,%lu*", (unsigned long)seq++,
                     (unsigned long)c[0], (unsigned long)c[1], (unsigned long)c[2]);
    uint8_t x = linkChecksum(buf, n);
    snprintf(buf + n, sizeof(buf) - n, "%02X\n", x);
    Serial2.print(buf);
  }

  if (watch && (int32_t)(now - nextWatch) >= 0) {
    nextWatch = now + 500;
    uint32_t c[3];
    snapshot(c);
    Serial.printf("FS1 %lu (%lu Hz)  FS2 %lu (%lu Hz)  FS3 %lu (%lu Hz)\n",
                  (unsigned long)c[0], (unsigned long)((c[0] - prevWatch[0]) * 2),
                  (unsigned long)c[1], (unsigned long)((c[1] - prevWatch[1]) * 2),
                  (unsigned long)c[2], (unsigned long)((c[2] - prevWatch[2]) * 2));
    for (int i = 0; i < 3; i++) prevWatch[i] = c[i];
  }

  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    if (cmd == "show") {
      uint32_t c[3];
      snapshot(c);
      Serial.printf("FS1 %lu  FS2 %lu  FS3 %lu\n", (unsigned long)c[0], (unsigned long)c[1], (unsign
    } else if (cmd == "watch") {
      watch = !watch;
      Serial.printf("watch %s\n", watch ? "on" : "off");
    } else if (cmd.length()) {
      Serial.println(F("commands: show, watch"));
    }
  }
}
```

## src/controller/main.cpp (ESP32 #1)

```
// ESP32 #1 (U2) · controller board · Team 13 Chocolate batch plant
//
```

```
// Drives SV1..SV4 and pumps P1..P4, reads the START switch, receives flow-sensor counts from
// ESP32 #2 (U7) over UART and runs the whole batch:
//     dose line by line (fast, then slow for the last part, stop early by the learned overrun)
//  -> water chase into the mixing tank -> report -> wait for the hand valves to be switched
//  -> second START press: water flush to the 2 L jug.
//
// Line numbers used everywhere: 1 = Red (P1, FT1, SV1), 2 = Yellow (P2, FT2, SV2), 3 = Green (P3, F
// 4 = Flush water (P4, SV4). The header outlet is split by two HAND ball valves the firmware cannot
//     BV-M (header -> mixing tank) and BV-J (header -> flush jug).
//     Batch and chase: BV-M open, BV-J shut.   Flush, prime, calibration: BV-M shut, BV-J open.
// Type "help" in the serial monitor (115200 baud, newline line ending) for every command.

#include <Arduino.h>
#include <Preferences.h>
#include "link.h"

// ------------------------------------------------------------------ pins (PCB sheets 2-5)
static const uint8_t SV_PIN[4]  = {23, 22, 21, 19};          // SV1..SV4, HIGH = coil on = valve ope
static const uint8_t PWM_PIN[4] = {32, 33, 25, 26};          // PWM_1..PWM_4 -> MDD10A DRV1 ch1/ch2,
static const uint8_t START_PIN  = 14;                        // ON/OFF_SWITCH1 to SGND, internal pul

static const uint32_t PWM_FREQ = 20000;   // MDD10A accepts up to 20 kHz; inaudible
static const uint8_t  PWM_BITS = 10;

static const char *LINE_NAME[4] = {"RED", "YELLOW", "GREEN", "FLUSH"};
enum { SV_FLUSH = 3 };
static const char *NEED_JUG = "to the JUG: BV-M shut, BV-J open";
static const char *NEED_MIX = "to the MIXING TANK: BV-M open, BV-J shut";

// ------------------------------------------------------------------ settings (saved in flash)
struct LineCfg {
  float k;        // pulses per mL, from calibration. 0 = not calibrated yet
  float target;   // mL for one batch
  uint8_t fast;   // % duty in the fast phase
  uint8_t slow;   // % duty in the slow phase (must still pump reliably)
  float overrun;  // mL that still arrive after the stop command; learned each dose
};
struct Config {
  LineCfg line[3];
  uint8_t order[3];     // dose order, 0-based line index
  float bandPct;        // slow phase covers the last bandPct % of the dose ...
  float bandMinMl;      // ... but never less than this
  uint8_t p4;           // flush pump duty %
  float p4Rate;         // mL/s the flush pump delivers at p4 duty (measured in the dry run)
  uint32_t chaseMs;     // water chase into the mixing tank after the last colour
  uint32_t flushMs;     // water flush to the jug
  bool learn;           // update overrun automatically after every dose
};
static Config cfg;
static Preferences prefs;

static void setDefaults() {
  // k = 0 forces a calibration before any metered dose.
  cfg.line[0] = {0.0f, 1800.0f, 100, 35, 3.0f};  // Red 60 %
```

```
  cfg.line[1] = {0.0f, 300.0f, 100, 35, 3.0f};   // Yellow 10 %
  cfg.line[2] = {0.0f, 900.0f, 100, 35, 3.0f};   // Green 30 %
  cfg.order[0] = 1; cfg.order[1] = 2; cfg.order[2] = 0;  // Yellow -> Green -> Red
  cfg.bandPct = 10.0f;
  cfg.bandMinMl = 20.0f;
  cfg.p4 = 60;
  cfg.p4Rate = 30.0f;
  cfg.chaseMs = 1500;
  cfg.flushMs = 6000;
  cfg.learn = true;
}
static void saveConfig() {
  prefs.begin("choc", false);
  prefs.putBytes("cfg", &cfg, sizeof(cfg));
  prefs.end();
  Serial.println(F("saved"));
}
static void loadConfig() {
  prefs.begin("choc", true);
  size_t n = prefs.getBytesLength("cfg");
  if (n == sizeof(cfg)) prefs.getBytes("cfg", &cfg, sizeof(cfg));
  else setDefaults();
  prefs.end();
}

// ------------------------------------------------------------------ outputs with interlocks
static bool valveOpen[4] = {false};
static uint8_t pumpPct[4] = {0};

static void writePump(int p, uint8_t pct) {
  pumpPct[p] = pct;
  ledcWrite(p, (uint32_t)pct * ((1u << PWM_BITS) - 1) / 100);
}
static void allOff() {
  for (int p = 0; p < 4; p++) writePump(p, 0);
  delay(100);  // let pumps spin down before valves shut (no deadheading)
  for (int v = 0; v < 4; v++) { digitalWrite(SV_PIN[v], LOW); valveOpen[v] = false; }
}
// A pump may only run when its own line valve is open. The outlet hand valves are the operator's jo
static bool pathOk(int p) { return valveOpen[p]; }

static bool setPump(int p, uint8_t pct) {
  if (pct > 100) pct = 100;
  if (pct > 0 && !pathOk(p)) {
    Serial.printf("! interlock: P%d needs SV%d open\n", p + 1, p + 1);
    return false;
  }
  writePump(p, pct);
  return true;
}
static bool setValve(int v, bool open) {
  if (!open && pumpPct[v]) {
    // closing a line valve first stops its pump, so it never runs against a closed valve
    writePump(v, 0);
```

```
    delay(100);
  }
  digitalWrite(SV_PIN[v], open ? HIGH : LOW);
  valveOpen[v] = open;
  return true;
}

// ------------------------------------------------------------- UART link from U7
static uint32_t counts[3] = {0, 0, 0};
static uint32_t lastFrameMs = 0, framesOk = 0, framesBad = 0;
static bool linkOk() { return framesOk > 0 && millis() - lastFrameMs < 150; }

static void pollLink() {
  static char buf[80];
  static uint8_t n = 0;
  while (Serial2.available()) {
    char c = Serial2.read();
    if (c == '\n') {
      buf[n] = 0;
      char *star = strchr(buf, '*');
      unsigned long seq, a, b, d;
      if (buf[0] == 'F' && star && sscanf(buf, "F,%lu,%lu,%lu,%lu", &seq, &a, &b, &d) == 4 &&
          (uint8_t)strtoul(star + 1, nullptr, 16) == linkChecksum(buf, star - buf + 1)) {
        counts[0] = a; counts[1] = b; counts[2] = d;
        lastFrameMs = millis();
        framesOk++;
      } else {
        framesBad++;
      }
      n = 0;
    } else if (c != '\r') {
      if (n < sizeof(buf) - 1) buf[n++] = c; else n = 0;
    }
  }
}

// ------------------------------------------------------------------ dose engine (one line at a time)
enum Phase { D_IDLE, D_OPEN, D_FAST, D_SLOW, D_STOP, D_SETTLE, D_DONE, D_FAULT };
static const char *PHASE_NAME[] = {"idle", "open", "fast", "slow", "stop", "settle", "done", "FAULT"

static const uint32_t VALVE_MS = 200;        // valve fully open before pump starts
static const uint32_t STOP_TO_CLOSE_MS = 150;  // pump stops, then line valve closes
static const uint32_t SETTLE_MS = 800;       // count late pulses after valve closes
static const uint32_t START_STALL_MS = 4000; // no pulses at all this long after start = fault
static const uint32_t STALL_MS = 2000;       // pulses stop mid-dose this long = fault
static const uint32_t MAX_DOSE_MS = 240000;

struct Dose {
  Phase ph = D_IDLE;
  int line = 0;
  float target = 0;       // mL; ignored in timed mode
  bool timed = false;     // calibration run: fixed duty for runMs, no metering
  uint32_t runMs = 0;
  uint8_t duty = 0;
```

```
  bool toMix = false;     // only for messages: the hand valves decide where it really goes
  uint32_t t0 = 0, tPh = 0, startCount = 0, stopCount = 0, lastPulseMs = 0, lastCount = 0;
  float mlAtStop = 0, mlFinal = 0;
  uint32_t pulsesFinal = 0;
  char fault[64] = "";
} dose;

static uint32_t lastPulses[3] = {0, 0, 0};  // pulses of the latest finished dose/calrun per line (f

static float mlOf(int line, uint32_t pulses) {
  return cfg.line[line].k > 0 ? pulses / cfg.line[line].k : 0;
}
static uint32_t dosePulses() { return counts[dose.line] - dose.startCount; }

static void doseFault(const char *why) {
  allOff();
  dose.ph = D_FAULT;
  strncpy(dose.fault, why, sizeof(dose.fault) - 1);
  Serial.printf("! FAULT on %s: %s. All pumps off, all valves closed.\n", LINE_NAME[dose.line], why)
}

static bool doseBegin(int line, float target, bool toMix, bool timed, uint32_t runMs, uint8_t duty)
  if (!linkOk()) { Serial.println(F("! no data from sensor board U7, check UART wiring and that U7 i
  if (!timed && cfg.line[line].k <= 0) { Serial.printf("! %s not calibrated (k=0). Run calrun/k firs
  dose = Dose();
  dose.line = line; dose.target = target; dose.timed = timed; dose.runMs = runMs;
  dose.duty = duty; dose.toMix = toMix;
  setValve(line, true);
  dose.startCount = dose.lastCount = counts[line];
  dose.t0 = dose.tPh = dose.lastPulseMs = millis();
  dose.ph = D_OPEN;
  if (timed) Serial.printf("> run %s %lu ms at %u%% (%s)\n", LINE_NAME[line], (unsigned long)runMs,
  else Serial.printf("> dose %s %.1f mL (%s)\n", LINE_NAME[line], target, toMix ? NEED_MIX : NEED_JU
  return true;
}

static void doseTelemetry(uint32_t now) {
  static uint32_t nextT = 0, prevCount = 0, prevMs = 0;
  if (dose.ph < D_FAST || dose.ph > D_SETTLE || (int32_t)(now - nextT) < 0) return;
  nextT = now + 250;
  uint32_t p = dosePulses();
  float rate = (prevMs && now > prevMs && p >= prevCount) ? mlOf(dose.line, p - prevCount) * 1000.0f
  prevCount = p; prevMs = now;
  // CSV: D,t_ms,line,phase,duty%,pulses,mL,mL_per_s
  Serial.printf("D,%lu,%d,%s,%u,%lu,%.1f,%.1f\n", (unsigned long)(now - dose.t0), dose.line + 1, PHA
                pumpPct[dose.line], (unsigned long)p, mlOf(dose.line, p), rate);
}

static void doseStep() {
  if (dose.ph == D_IDLE || dose.ph == D_DONE || dose.ph == D_FAULT) return;
  uint32_t now = millis();
  LineCfg &L = cfg.line[dose.line];
  uint32_t c = counts[dose.line];
  if (c != dose.lastCount) { dose.lastCount = c; dose.lastPulseMs = now; }
```

```
  uint32_t p = dosePulses();
  float ml = mlOf(dose.line, p);

  if (dose.ph <= D_SLOW && !linkOk()) { doseFault("lost sensor link"); return; }
  if (now - dose.t0 > MAX_DOSE_MS) { doseFault("dose took too long"); return; }
  if (dose.ph == D_FAST || dose.ph == D_SLOW) {
    uint32_t limit = (p == 0) ? START_STALL_MS : STALL_MS;
    if (now - dose.lastPulseMs > limit) { doseFault(p == 0 ? "no flow after start (empty tank, air,
  }
  doseTelemetry(now);

  float band = max(dose.target * cfg.bandPct / 100.0f, cfg.bandMinMl);
  float stopAt = dose.target - L.overrun;

  switch (dose.ph) {
    case D_OPEN:
      if (now - dose.tPh >= VALVE_MS) {
        uint8_t d = dose.timed ? dose.duty : L.fast;
        if (!dose.timed && ml >= dose.target - band) d = L.slow;  // tiny dose: go straight to slow
        if (!setPump(dose.line, d)) { doseFault("interlock refused pump start"); return; }
        dose.ph = (d == L.slow && !dose.timed) ? D_SLOW : D_FAST;
        dose.tPh = now; dose.lastPulseMs = now;
      }
      break;
    case D_FAST:
      if (dose.timed) {
        if (now - dose.tPh >= dose.runMs) goto stop;
      } else if (ml >= stopAt) {
        goto stop;
      } else if (ml >= dose.target - band) {
        writePump(dose.line, L.slow);
        dose.ph = D_SLOW; dose.tPh = now;
      }
      break;
    case D_SLOW:
      if (ml >= stopAt) goto stop;
      break;
    case D_STOP:
      if (now - dose.tPh >= STOP_TO_CLOSE_MS) {
        setValve(dose.line, false);
        dose.ph = D_SETTLE; dose.tPh = now;
      }
      break;
    case D_SETTLE:
      if (now - dose.tPh >= SETTLE_MS && now - dose.lastPulseMs >= 300) {
        dose.pulsesFinal = p;
        dose.mlFinal = ml;
        lastPulses[dose.line] = p;
        if (dose.timed) {
          Serial.printf("= calrun %s: %lu pulses in %lu ms (%.1f Hz). Now weigh the jug and type: k
                        LINE_NAME[dose.line], (unsigned long)p, (unsigned long)dose.runMs,
                        p * 1000.0f / dose.runMs, dose.line + 1);
        } else {
          float coast = dose.mlFinal - dose.mlAtStop;
```

```
          if (cfg.learn && coast >= 0 && coast < 50) L.overrun = 0.5f * L.overrun + 0.5f * coast;
          Serial.printf("= %s target %.1f mL, metered %.1f mL (error %+.1f mL, %+.2f %%), coast %.1f
                        LINE_NAME[dose.line], dose.target, dose.mlFinal, dose.mlFinal - dose.target,
                        (dose.mlFinal - dose.target) * 100.0f / dose.target, coast, L.overrun, (unsi
        }
        dose.ph = D_DONE;
      }
      break;
    default:
      break;
  }
  return;
stop:
  writePump(dose.line, 0);
  dose.mlAtStop = ml;
  dose.stopCount = c;
  dose.ph = D_STOP; dose.tPh = now;
}

// ------------------------------------------------------------------ simple timed flush-pump runs
struct Timed { bool on = false; uint32_t t0 = 0, ms = 0; int phase = 0; bool toMix = false; } p4run;
static float jugMl = 0;  // estimated water in the jug since it was last emptied

static bool p4Begin(bool toMix, uint32_t ms) {
  if (!toMix && jugMl + cfg.p4Rate * ms / 1000.0f > 1800.0f) {
    Serial.printf("! jug would overflow (now ~%.0f mL). Empty it, type 'jug 0', then 'flush'.\n", ju
    return false;
  }
  setValve(SV_FLUSH, true);
  p4run = Timed();
  p4run.on = true; p4run.t0 = millis(); p4run.ms = ms; p4run.toMix = toMix;
  Serial.printf("> P4 water %lu ms, %s (%s)\n", (unsigned long)ms, toMix ? "chase" : "flush", toMix
  return true;
}
static void p4Step() {
  if (!p4run.on) return;
  uint32_t now = millis();
  switch (p4run.phase) {
    case 0:
      if (now - p4run.t0 >= VALVE_MS) {
        if (!setPump(3, cfg.p4)) { allOff(); p4run.on = false; return; }
        p4run.phase = 1; p4run.t0 = now;
      }
      break;
    case 1:
      if (now - p4run.t0 >= p4run.ms) { writePump(3, 0); p4run.phase = 2; p4run.t0 = now; }
      break;
    case 2:
      if (now - p4run.t0 >= STOP_TO_CLOSE_MS) {
        setValve(SV_FLUSH, false);
        if (!p4run.toMix) jugMl += cfg.p4Rate * p4run.ms / 1000.0f;
        p4run.on = false;
        Serial.printf("= P4 done. Jug estimate ~%.0f mL\n", jugMl);
      }
```

```
      break;
  }
}

// ------------------------------------------------------------- batch sequence
enum Batch { B_IDLE, B_LINE, B_CHASE, B_WAIT_FLUSH, B_FLUSH, B_DONE, B_ABORTED };
static Batch batch = B_IDLE;
static int batchIdx = 0;
static float result[3];
static uint32_t batchT0 = 0;

static bool busy() { return batch == B_LINE || batch == B_CHASE || batch == B_FLUSH || p4run.on ||
                            (dose.ph != D_IDLE && dose.ph != D_DONE && dose.ph != D_FAULT); }

static void batchAbort(const char *why) {
  allOff();
  p4run.on = false;
  if (dose.ph != D_DONE && dose.ph != D_IDLE) dose.ph = D_FAULT;
  batch = B_ABORTED;
  Serial.printf("! BATCH ABORTED: %s\n", why);
}

static void batchStart() {
  if (busy()) { Serial.println(F("! busy")); return; }
  for (int i = 0; i < 3; i++)
    if (cfg.line[i].k <= 0) { Serial.printf("! %s not calibrated, batch refused\n", LINE_NAME[i]); r
  if (!linkOk()) { Serial.println(F("! no data from sensor board U7, batch refused")); return; }
  allOff();
  Serial.printf("\n>>> BATCH START (BV-M open, BV-J shut)  order %s -> %s -> %s, then chase %lu ms,
                LINE_NAME[cfg.order[0]], LINE_NAME[cfg.order[1]], LINE_NAME[cfg.order[2]],
                (unsigned long)cfg.chaseMs, (unsigned long)cfg.flushMs);
  batchT0 = millis();
  batchIdx = 0;
  int l = cfg.order[0];
  if (!doseBegin(l, cfg.line[l].target, true, false, 0, 0)) { batchAbort("first dose would not start
  batch = B_LINE;
}

static void batchReport() {
  Serial.println(F("\n===== BATCH REPORT ====="));
  float tot = 0;
  for (int i = 0; i < 3; i++) {
    int l = cfg.order[i];
    tot += result[l];
    Serial.printf("%-7s target %7.1f mL  metered %7.1f mL  error %+6.1f mL (%+.2f %%)\n", LINE_NAME[
                  cfg.line[l].target, result[l], result[l] - cfg.line[l].target,
                  (result[l] - cfg.line[l].target) * 100.0f / cfg.line[l].target);
  }
  Serial.printf("colour total %.1f mL, + chase water ~%.0f mL, time %.1f s\n", tot,
                cfg.p4Rate * cfg.chaseMs / 1000.0f, (millis() - batchT0) / 1000.0f);
  for (int i = 0; i < 3; i++) {
    int l = cfg.order[i];
    Serial.printf("ratio %s %.1f %%  ", LINE_NAME[l], tot > 0 ? result[l] * 100.0f / tot : 0);
  }
```

```
  Serial.println(F("\n========================"));
}

static void batchStep() {
  switch (batch) {
    case B_LINE:
      if (dose.ph == D_FAULT) { batchAbort(dose.fault); break; }
      if (dose.ph == D_DONE) {
        result[dose.line] = dose.mlFinal;
        if (++batchIdx < 3) {
          int l = cfg.order[batchIdx];
          if (!doseBegin(l, cfg.line[l].target, true, false, 0, 0)) batchAbort("next dose would not
        } else {
          // BV-M is still open: push the last colour out of the header into the mixing tank
          if (cfg.chaseMs == 0) batch = B_CHASE;
          else if (p4Begin(true, cfg.chaseMs)) batch = B_CHASE;
          else batchAbort("chase would not start");
        }
      }
      break;
    case B_CHASE:
      if (!p4run.on) {
        batchReport();
        batch = B_WAIT_FLUSH;
        Serial.println(F(">>> BATCH COMPLETE. To flush: SHUT BV-M, OPEN BV-J, then press START (or t
      }
      break;
    case B_FLUSH:
      if (!p4run.on) {
        batch = B_DONE;
        Serial.println(F(">>> FLUSH DONE. Before the next batch: SHUT BV-J, OPEN BV-M. Empty the jug
      }
      break;
    default:
      break;
  }
}

static void batchFlush() {
  if (busy()) { Serial.println(F("! busy")); return; }
  if (cfg.flushMs == 0 || !p4Begin(false, cfg.flushMs)) { Serial.println(F("! flush not started"));
  batch = B_FLUSH;
}

// ------------------------------------------------------------------ start switch
static void pollStart() {
  static bool last = true, stable = true;
  static uint32_t tChange = 0;
  bool r = digitalRead(START_PIN);
  if (r != last) { last = r; tChange = millis(); }
  if (millis() - tChange > 40 && r != stable) {
    stable = r;
    if (!stable) {  // pressed (pulled to SGND)
      if (busy()) batchAbort("START pressed during run (used as stop)");
```

```
      else if (batch == B_WAIT_FLUSH) batchFlush();
      else batchStart();
    }
  }
}

// ------------------------------------------------------------- serial commands
static void printConfig() {
  Serial.println(F("line    k(p/mL)  target  fast%  slow%  overrun"));
  for (int i = 0; i < 3; i++)
    Serial.printf("%d %-6s %7.3f %7.1f %5u %6u %8.1f\n", i + 1, LINE_NAME[i], cfg.line[i].k, cfg.lin
                  cfg.line[i].fast, cfg.line[i].slow, cfg.line[i].overrun);
  Serial.printf("order %d%d%d  band %.1f %% (min %.0f mL)  learn %d\n", cfg.order[0] + 1, cfg.order[
                cfg.bandPct, cfg.bandMinMl, cfg.learn);
  Serial.printf("p4 %u %%  p4rate %.1f mL/s  chase %lu ms  flush %lu ms  jug ~%.0f mL\n", cfg.p4, cf
                (unsigned long)cfg.chaseMs, (unsigned long)cfg.flushMs, jugMl);
}
static void printStatus() {
  Serial.printf("link %s (ok %lu, bad %lu)  counts %lu %lu %lu  start=%s\n", linkOk() ? "OK" : "DOWN
                (unsigned long)framesOk, (unsigned long)framesBad, (unsigned long)counts[0], (unsign
                (unsigned long)counts[2], digitalRead(START_PIN) ? "up" : "PRESSED");
  Serial.print(F("valves "));
  for (int v = 0; v < 4; v++) Serial.printf("SV%d=%d ", v + 1, valveOpen[v]);
  Serial.print(F(" pumps "));
  for (int p = 0; p < 4; p++) Serial.printf("P%d=%u%% ", p + 1, pumpPct[p]);
  static const char *BN[] = {"idle", "dosing", "chase", "WAITING FOR FLUSH (shut BV-M, open BV-J, pr
  Serial.printf("\nbatch %s  dose %s\n", BN[batch], PHASE_NAME[dose.ph]);
}
static void help() {
  Serial.println(F(
      "--- Team 13 Chocolate controller (line 1=RED 2=YELLOW 3=GREEN 4=FLUSH) ---\n"
      "start | abort          run / stop the full batch (same as the START switch)\n"
      "                       after a batch, the next START press runs the flush\n"
      "  hand valves: batch/chase = BV-M open, BV-J shut; everything else = BV-M shut, BV-J open\n"
      "status | show          live I/O state | settings\n"
      "prime <1-4> <sec>      run a line into the JUG at fast duty (fill lines, purge air)\n"
      "calrun <1-3> <sec> [%] timed run into the JUG, counts pulses (calibration)\n"
      "k <1-3> <mL>           set k = pulses of last calrun/dose / mL weighed\n"
      "dose <1-3> <mL> [mix]  metered dose with full fast/slow profile (default to JUG)\n"
      "chase | flush          water into mixing tank | water flush to jug\n"
      "jug <mL>               set jug estimate (jug 0 after emptying)\n"
      "valve <1-4> <0|1>      manual valve, idle only\n"
      "pump <1-4> <%>         manual pump, idle only, interlocked\n"
      "off                    everything off\n"
      "set <key> <value>      keys: k1..3 tgt1..3 fast1..3 slow1..3 ovr1..3 order(e.g. 231)\n"
      "                       band bandmin p4 p4rate chase flush learn\n"
      "save | load | defaults settings in flash"));
}

static void handleCommand(String line) {
  line.trim();
  if (!line.length()) return;
  char buf[96];
  line.toCharArray(buf, sizeof(buf));
```

```
  char *cmd = strtok(buf, " ");
  char *a1 = strtok(nullptr, " ");
  char *a2 = strtok(nullptr, " ");
  char *a3 = strtok(nullptr, " ");
  String c(cmd);
  c.toLowerCase();
  int n = a1 ? atoi(a1) : 0;

  if (c == "help" || c == "?") help();
  else if (c == "status") printStatus();
  else if (c == "show") printConfig();
  else if (c == "start") { if (batch == B_WAIT_FLUSH) batchFlush(); else batchStart(); }
  else if (c == "abort" || c == "stop") batchAbort("abort command");
  else if (c == "off") { allOff(); p4run.on = false; dose.ph = D_IDLE; batch = B_IDLE; Serial.printl
  else if (busy()) Serial.println(F("! busy, type abort first"));
  else if (c == "prime" && n >= 1 && n <= 4 && a2) {
    uint32_t ms = (uint32_t)(atof(a2) * 1000);
    if (n == 4) p4Begin(false, ms);
    else doseBegin(n - 1, 0, false, true, ms, cfg.line[n - 1].fast);
  } else if (c == "calrun" && n >= 1 && n <= 3 && a2) {
    uint8_t duty = a3 ? atoi(a3) : cfg.line[n - 1].fast;
    doseBegin(n - 1, 0, false, true, (uint32_t)(atof(a2) * 1000), duty);
  } else if (c == "k" && n >= 1 && n <= 3 && a2) {
    float ml = atof(a2);
    if (ml <= 0 || lastPulses[n - 1] == 0) Serial.println(F("! need a calrun or dose on that line fi
    else {
      cfg.line[n - 1].k = lastPulses[n - 1] / ml;
      Serial.printf("k%d = %lu / %.1f = %.4f pulses/mL (type save to keep)\n", n, (unsigned long)las
    }
  } else if (c == "dose" && n >= 1 && n <= 3 && a2) {
    bool mix = a3 && String(a3) == "mix";
    doseBegin(n - 1, atof(a2), mix, false, 0, 0);
  } else if (c == "chase") p4Begin(true, cfg.chaseMs);
  else if (c == "flush") { if (batch == B_WAIT_FLUSH) batchFlush(); else p4Begin(false, cfg.flushMs)
  else if (c == "jug" && a1) { jugMl = atof(a1); Serial.printf("jug ~%.0f mL\n", jugMl); }
  else if (c == "valve" && n >= 1 && n <= 4 && a2) { setValve(n - 1, atoi(a2)); printStatus(); }
  else if (c == "pump" && n >= 1 && n <= 4 && a2) { setPump(n - 1, atoi(a2)); printStatus(); }
  else if (c == "save") saveConfig();
  else if (c == "load") { loadConfig(); printConfig(); }
  else if (c == "defaults") { setDefaults(); printConfig(); Serial.println(F("(not saved yet)")); }
  else if (c == "set" && a1 && a2) {
    String k(a1);
    k.toLowerCase();
    float v = atof(a2);
    int i = k.length() > 0 ? k.charAt(k.length() - 1) - '1' : -1;
    String base = k.substring(0, k.length() - 1);
    bool ok = true;
    if (i >= 0 && i < 3 && base == "k") cfg.line[i].k = v;
    else if (i >= 0 && i < 3 && base == "tgt") cfg.line[i].target = v;
    else if (i >= 0 && i < 3 && base == "fast") cfg.line[i].fast = constrain((int)v, 0, 100);
    else if (i >= 0 && i < 3 && base == "slow") cfg.line[i].slow = constrain((int)v, 0, 100);
    else if (i >= 0 && i < 3 && base == "ovr") cfg.line[i].overrun = v;
    else if (k == "order" && strlen(a2) == 3) {
      uint8_t o[3]; bool seen[3] = {false, false, false};
```

```
      for (int j = 0; j < 3; j++) { o[j] = a2[j] - '1'; if (o[j] > 2 || seen[o[j]]) ok = false; else
      if (ok) memcpy(cfg.order, o, 3);
    } else if (k == "band") cfg.bandPct = v;
    else if (k == "bandmin") cfg.bandMinMl = v;
    else if (k == "p4") cfg.p4 = constrain((int)v, 0, 100);
    else if (k == "p4rate") cfg.p4Rate = v;
    else if (k == "chase") cfg.chaseMs = (uint32_t)v;
    else if (k == "flush") cfg.flushMs = (uint32_t)v;
    else if (k == "learn") cfg.learn = v != 0;
    else ok = false;
    if (ok) { printConfig(); Serial.println(F("(type save to keep)")); }
    else Serial.println(F("! unknown key or bad value"));
  } else Serial.println(F("? type help"));
}

// ------------------------------------------------------------- setup / loop
void setup() {
  // Valves first: NC valves stay shut while the board boots.
  for (int v = 0; v < 4; v++) { pinMode(SV_PIN[v], OUTPUT); digitalWrite(SV_PIN[v], LOW); }
  for (int p = 0; p < 4; p++) { ledcSetup(p, PWM_FREQ, PWM_BITS); ledcAttachPin(PWM_PIN[p], p); ledc
  pinMode(START_PIN, INPUT_PULLUP);
  Serial.begin(115200);
  Serial2.begin(LINK_BAUD, SERIAL_8N1, LINK_RX_PIN, LINK_TX_PIN);
  loadConfig();
  delay(300);
  Serial.println(F("\n[U2 controller] Team 13 Chocolate. Type help.\nFor a batch: BV-M OPEN, BV-J SH
  printConfig();
}

void loop() {
  pollLink();
  pollStart();
  doseStep();
  p4Step();
  batchStep();
  if (Serial.available()) handleCommand(Serial.readStringUntil('\n'));
}
```